

INDIVIDUAL LAB - WORK RECORD

MODULE 1

Selenium WebDriver Automation

Student Name: Protyusha Mondal

Roll No.: 12023052002072

Assignments: 1, 2, 3, 4, 5 and 6

INDIVIDUAL LAB - WORK RECORD

Lab Report: Assignment 1 – The Multi-Locator Challenge

Field	Details
Tier	Tier 1: Fundamentals & Basic Interactivity
Module	Assignment 1 – Web Element Locators
Framework	Selenium WebDriver with Python
Target URL	https://www.saucedemo.com/

1. Objectives

- Interoperably utilize multiple Selenium locator strategies (By.ID, By.NAME, By.XPATH) to identify distinct web elements on a login interface.
- Automate basic form submission actions (.clear(), .send_keys(), .click()).
- Programmatically validate successful page navigation using Python assertion statements against driver.current_url.

2. Technical Task Execution Summary

Target Element	Applied Locator Strategy	Code Pattern
Username Input	By.ID	driver.find_element(By.ID, "user-name")
Password Input	By.NAME	driver.find_element(By.NAME, "password")
Login Button	By.XPATH	driver.find_element(By.XPATH, "//input[@id='login-button']")
URL Assertion	URL Pattern Matching	assert "/inventory.html" in driver.current_url

3. Source Code & Screenshots

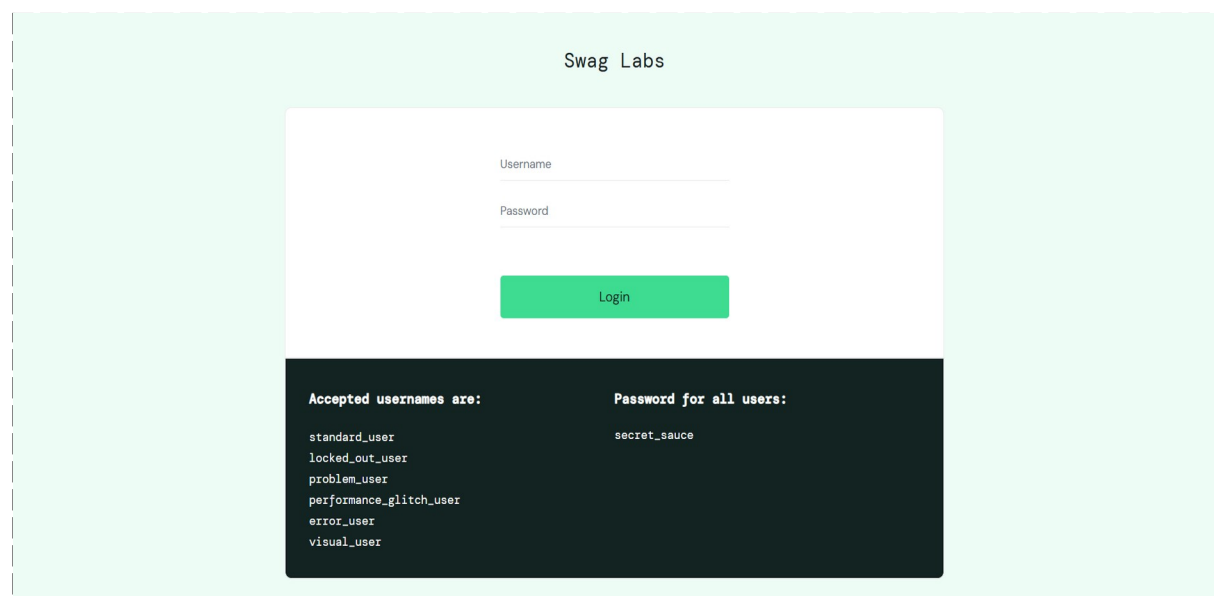
The automation script (assignment_1_locators.py) is divided into a setup block, the multi-locator interaction block, and the final URL validation block. Each screenshot is placed after the code that captures it.

3.1 Setup & Login Page Screenshot

```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

# 1. Initialize Driver
driver = webdriver.Chrome()
driver.get("https://www.saucedemo.com/")
driver.maximize_window()
time.sleep(1)

# SCREENSHOT 1: Login Page Overview
driver.save_screenshot("assignment_1_step1_login_page.png")
print("Saved: assignment_1_step1_login_page.png")
```

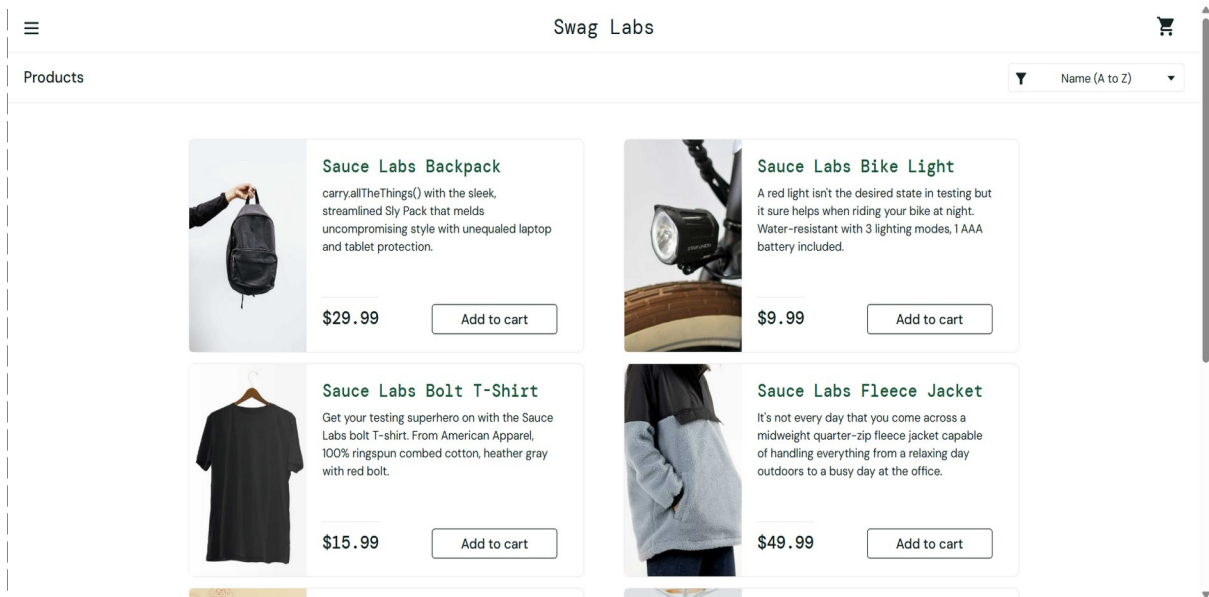


3.2 Multi-Locator Strategies

```
# =====  
# MULTI-LOCATOR STRATEGIES  
# =====  
  
# 1. Interact with Username Field using By.ID  
username_field = driver.find_element(By.ID, "user-name")  
username_field.clear()  
username_field.send_keys("standard_user")  
print("Username entered using By.ID")  
  
# 2. Interact with Password Field using By.NAME  
password_field = driver.find_element(By.NAME, "password")  
password_field.clear()  
password_field.send_keys("secret_sauce")  
print("Password entered using By.NAME")  
  
# 3. Interact with Login Button using By.XPATH  
login_button = driver.find_element(By.XPATH, "//input[@id='login-button']")  
login_button.click()  
print("Login button clicked using By.XPATH")  
  
time.sleep(2)
```

3.3 Validation: Assert Resulting URL

```
# =====  
# VALIDATION: ASSERT RESULTING URL  
# =====  
  
current_url = driver.current_url  
print("Current Page URL:", current_url)  
  
# Assertion to verify login success  
assert "/inventory.html" in current_url, f"Assertion Failed: Expected '/inventory.html' in URL, but got '{current_url}'"  
print("Validation Passed: URL contains '/inventory.html' successfully!")  
  
# SCREENSHOT 2: Inventory Page After Login  
driver.save_screenshot("assignment_1_step2_inventory_page.png")  
print("Saved: assignment_1_step2_inventory_page.png")  
  
driver.quit()
```



4. Execution & Verification Steps

- **Execution:** Execute command `python assignment_1_locators.py` in the active virtual environment.

Terminal Output Verification:

- Entered credentials into user-name and password fields.
- Triggered submit click via XPath selector.
- Captured current URL: `https://www.saucedemo.com/inventory.html`.
- Python assertion evaluated True without raising an `AssertionError`.

Screenshot Proof Output:

- `assignment_1_step1_login_page.png`: Captures initial login interface state.
- `assignment_1_step2_inventory_page.png`: Captures inventory screen post-authentication.

5. Conclusion

Assignment 1 from Tier 1 was successfully implemented. The script demonstrates locator identification across `By.ID`, `By.NAME`, and `By.XPATH`, culminating in a validated assertion check confirming navigation to the inventory page.

Lab Report: Assignment 2 – Synchronization & Explicit Waits

Field	Details
Tier	Tier 1: Fundamentals & Basic Interactivity
Module	Assignment 2 – Synchronization & Explicit Waits
Framework	Selenium WebDriver with Python
Target URL	https://the-internet.herokuapp.com/dynamic_loading/1

1. Objectives

- Master test script synchronization techniques for dynamic web applications and AJAX elements.
- Enforce strict compliance against non-deterministic sleep calls (time.sleep()).
- Utilize WebDriverWait combined with expected_conditions to wait for DOM element visibility before text extraction.

2. Technical Task Execution Summary

Constraint / Task	Applied Solution	Implementation Code
No time.sleep()	Strict WebDriverWait explicit synchronization	wait = WebDriverWait(driver, 10)
Button Interaction	Wait until element is clickable	EC.element_to_be_clickable((By.XPATH, "..."))
Dynamic AJAX Load	Wait for element visibility	EC.visibility_of_element_located((By.XPATH, "..."))
Text Verification	Assert extracted text	assert extracted_text == "Hello World!"

3. Source Code & Screenshots

The automation script (assignment_2_waits.py) is divided into a setup block and the explicit wait implementation block. Each screenshot is placed after the code that captures it.

3.1 Setup & Initial Page Screenshot

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# 1. Initialize Driver
driver = webdriver.Chrome()
driver.get("https://the-internet.herokuapp.com/dynamic_loading/1")
driver.maximize_window()

# 2. Configure Explicit Wait (Max timeout: 10 seconds)
wait = WebDriverWait(driver, 10)

# SCREENSHOT 1: Initial Page State
driver.save_screenshot("assignment_2_step1_initial_page.png")
print("Saved: assignment_2_step1_initial_page.png")
```

Dynamically Loaded Page Elements

Example 1: Element on page that is hidden

Start

Powered by [Elemental Selenium](#)

Find me on [GitHub](#)

3.2 Explicit Wait Implementation (No time.sleep)

```
# =====  
# EXPLICIT WAIT IMPLEMENTATION (NO TIME.SLEEP)  
# =====  
  
# Locate and click the Start button  
start_button = wait.until(  
    EC.element_to_be_clickable((By.XPATH, "//div[@id='start']/button"))  
)  
start_button.click()  
print("Start button clicked. Waiting for dynamic text to load...")  
  
# Explicitly wait until the dynamically loaded text element is VISIBLE  
# Constraint Verified: Zero usage of time.sleep()  
finish_element = wait.until(  
    EC.visibility_of_element_located((By.XPATH, "//div[@id='finish']/h4"))  
)  
  
# Extract and print the loaded text  
extracted_text = finish_element.text  
print(f"Dynamically Loaded Text Successfully Extracted: '{extracted_text}'")  
  
# Assertion Validation  
assert extracted_text == "Hello World!", f"Expected 'Hello World!', but got '{extracted_text}'"  
print("Validation Passed: Extracted text matches expected value!")  
  
# SCREENSHOT 2: Revealed Dynamic Text  
driver.save_screenshot("assignment_2_step2_dynamic_text_revealed.png")  
print("Saved: assignment_2_step2_dynamic_text_revealed.png")  
  
driver.quit()
```

Dynamically Loaded Page Elements

Example 1: Element on page that is hidden

Hello World!

Powered by [Elemental Selenium](#)

For more on GitHub

4. Execution & Verification Steps

- **Execution:** Run `python assignment_2_waits.py` inside the active virtual environment.

Terminal Output Verification:

- Clicked "Start" button on dynamic loading page.
- WebDriverWait polled the DOM during AJAX delay and successfully resumed execution upon element rendering.
- Extracted dynamic text: "Hello World!".
- Assertion evaluated True.

Screenshot Proof Output:

- assignment_2_step1_initial_page.png: Captures initial page before triggering dynamic load.
- assignment_2_step2_dynamic_text_revealed.png: Captures loaded dynamic text after explicit wait resolution.

5. Conclusion

Assignment 2 from Tier 1 was successfully completed. Dynamic content delayed by background execution was reliably synchronized using WebDriverWait and expected_conditions, ensuring fast and reliable execution without relying on time.sleep()).

Lab Report: Assignment 3 – Dynamic Dropdowns & Checkboxes

Field	Details
Tier	Tier 1: Fundamentals & Basic Interactivity
Module	Assignment 3 – Dynamic Elements, Checkboxes & Autocomplete Dropdowns
Framework	Selenium WebDriver with Python
Target URL	https://testautomationpractice.blogspot.com/

1. Objectives

- Automate state-based element selection using `.click()` and `.is_selected()`.
- Programmatically verify checkbox states using boolean assertions.
- Interact with dynamic search-as-you-type input fields, wait for suggestion lists, and loop through dynamic elements to select specific options.

2. Technical Task Execution Summary

Task Requirement	Implementation Approach	Code Pattern
Checkbox Selection	Conditional click based on current state	<code>if not cb.is_selected(): cb.click()</code>
State Verification	State check via <code>.is_selected()</code>	<code>assert sunday_cb.is_selected() == True</code>
Search Autocomplete	Text injection into search-as-you-type box	<code>search_input.send_keys("Selenium")</code>
Suggestion Looping	Iterate dynamic list & click target string	<code>for opt in suggestions: if opt.text == target: opt.click()</code>

3. Source Code & Screenshots

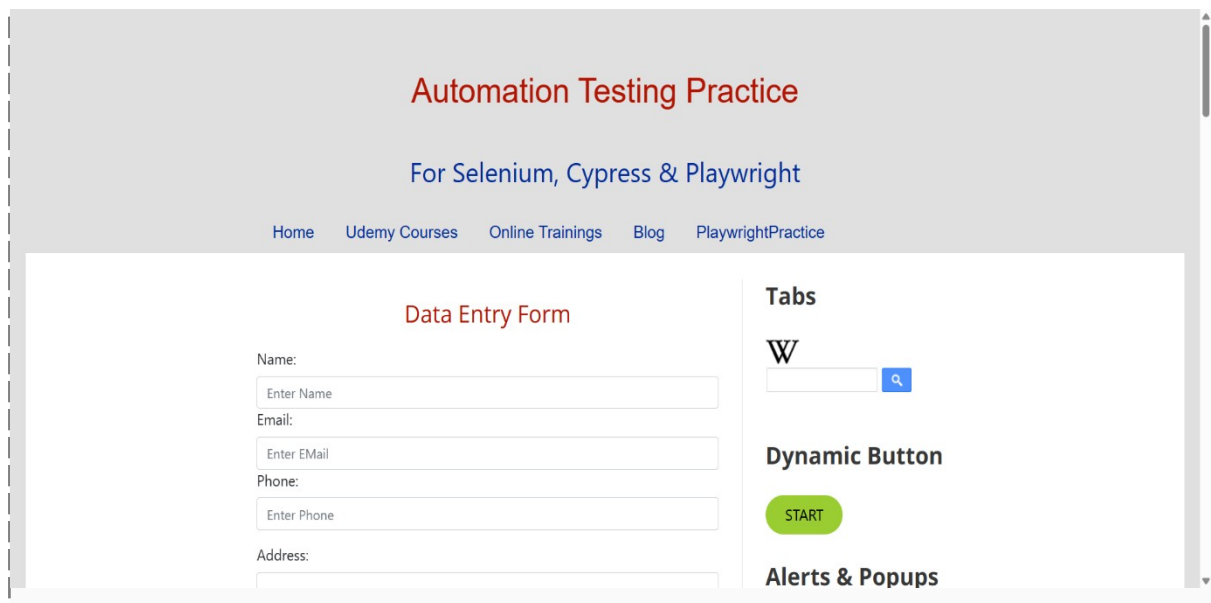
The automation script (`assignment_3_dropdowns_checkboxes.py`) is divided into a setup block and two tasks. Each part is followed by the screenshot it captures during execution.

3.1 Setup & Initial Page Screenshot

```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# 1. Initialize Driver
driver = webdriver.Chrome()
driver.get("https://testautomationpractice.blogspot.com/")
driver.maximize_window()
wait = WebDriverWait(driver, 10)

# SCREENSHOT 1: Initial Page Overview
driver.save_screenshot("assignment_3_step1_initial_page.png")
print("Saved: assignment_3_step1_initial_page.png")
```



3.2 Task 1 - Checkbox Selection & Verification (.is_selected())

```
# =====
# TASK 1: CHECKBOX SELECTION & VERIFICATION (.is_selected())
# =====

# Scroll to Checkbox section
sunday_cb = driver.find_element(By.ID, "sunday")
monday_cb = driver.find_element(By.ID, "monday")

driver.execute_script("arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", sunday_cb)
time.sleep(1)

# Select 'Sunday' if not already selected
if not sunday_cb.is_selected():
    sunday_cb.click()

# Verify state using .is_selected()
assert sunday_cb.is_selected() == True, "Verification Failed: Sunday checkbox should be selected!"
assert monday_cb.is_selected() == False, "Verification Failed: Monday checkbox should NOT be selected!"

print("Checkbox Selection & State Verification (.is_selected()) Passed!")

# SCREENSHOT 2: Selected Checkboxes
driver.save_screenshot("assignment_3_step2_checkboxes_selected.png")
print("Saved: assignment_3_step2_checkboxes_selected.png")
```

The image shows a web form on the left and a sidebar menu on the right. The form contains the following fields and options:

- Enter Email:
- Phone:
- Address:
- Gender: ☐ Male ☐ Female
- Days: ☒ Sunday ☐ Monday ☐ Tuesday ☐ Wednesday ☐ Thursday ☐ Friday ☐ Saturday
- Country:
- Colors:
- Sorted List:

The sidebar menu on the right contains the following items:

- Dynamic Button
- START
- Alerts & Popups
 - Simple Alert
 - Confirmation Alert
 - Prompt Alert
- New Tab
- Popup Windows
- Mouse Hover
 - Move the mouse over the button to open the dropdown menu.

3.3 Task 2 – Autocomplete / Search-As-You-Type Dropdown

```
# =====
# TASK 2: AUTOCOMPLETE / SEARCH-AS-YOU-TYPE DROPDOWN
# =====
# Locate Wikipedia search input box (autocomplete pattern)
search_input = driver.find_element(By.ID, "Wikipedia1_wikipedia-search-input")
search_button = driver.find_element(By.CLASS_NAME, "wikipedia-search-button")

driver.execute_script("arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", search_input)
time.sleep(1)

# Type into autocomplete search field
search_term = "Selenium"
target_option = "Selenium (software)"

search_input.clear()
search_input.send_keys(search_term)
search_button.click()

# Wait for dynamic suggestions to populate in DOM
wait.until(EC.presence_of_element_located((By.XPATH, "//div[@id='wikipedia-search-result-link']/a")))

# Loop through dynamic suggested results to select matching option
suggestions = driver.find_elements(By.XPATH, "//div[@id='wikipedia-search-result-link']/a")
print(f"Found {len(suggestions)} autocomplete suggestions for '{search_term}':")

matched = False
for option in suggestions:
    option_text = option.text
    print(f" - Suggestion: {option_text}")

    if option_text.strip().lower() == target_option.lower():
        print(f"Match found! Clicking option: '{option_text}'")
        option.click()
        matched = True
        break

assert matched == True, f"Failed to find matching option '{target_option}' in dynamic suggestions list!"

time.sleep(2)

# Switch focus to new tab opened by suggestion click if applicable
if len(driver.window_handles) > 1:
    driver.switch_to.window(driver.window_handles[1])

# SCREENSHOT 3: Selected Dynamic Dropdown Result
driver.save_screenshot("assignment_3_step3_dropdown_selected.png")
print("Saved: assignment_3_step3_dropdown_selected.png")

driver.quit()
```

Selenium (software)

21 languages

Contents

(Top)

History

Components

Selenium IDE

Selenium client API

Selenium Remote

Control

Selenium WebDriver

Python

Java

C#

Selenium Grid

See also

References

External links

Article Talk

Read

Edit

View history

From Wikipedia, the free encyclopedia



This article **needs more citations**. Please help improve this article by adding citations to reliable sources. Unsourced material may be challenged and removed.

Find sources: "Selenium" software · news · newspapers · books · scholar · JSTOR (January 2017) (Learn how and when to remove this message)

Selenium is an open source umbrella project for a range of tools and libraries aimed at supporting browser automation.^[3] It provides a playback tool for authoring functional tests across most modern web browsers, without the need to learn a test scripting language (Selenium IDE).^[4] It also provides a test domain-specific language (Selenese) to write tests in a number of popular programming languages, including JavaScript (Node.js), C#, Groovy, Java, Perl, PHP, Python, Ruby and Scala. Selenium runs on Windows, Linux, and macOS. It is open-source software released under the Apache License 2.0.

History

Selenium was originally developed by Jason Huggins in 2004 as an internal tool at ThoughtWorks.^[5] Huggins was later joined by other programmers and testers at

Selenium



Release	2004; 22 years ago
Stable release	4.49.0 ^[1] / 9 September 2026; 14 days ago
Written in	Selenium Server: Java; Selenium WebDriver (works without Selenium Server), official support: JavaScript (Node.js), Python, Ruby, Java or C# ^[2]
Operating system	Cross-platform
Type	Software testing framework

Appearance

Text

☐ Small

☒ Standard

☐ Large

Width

☒ Standard

☐ Wide

Color

☐ Automatic

☒ Light

☐ Dark

4. Execution & Verification Steps

- Execution:** Run python assignment_3_dropdowns_checkboxes.py in your terminal inside the virtual environment.

Terminal Output Verification:

- Checked Sunday checkbox and verified state returns True.
- Verified Monday checkbox returns False.
- Injected "Selenium" query into autocomplete input field.
- Dynamically parsed returned list items and located matching option "Selenium (software)".

Screenshot Proof Output:

- assignment_3_step1_initial_page.png: Initial page overview before interaction.
- assignment_3_step2_checkboxes_selected.png: Verified state of selected checkbox.
- assignment_3_step3_dropdown_selected.png: Final state showing selected dynamic suggestion element.

5. Conclusion

Assignment 3 from Tier 1 was successfully implemented. The script demonstrates checkbox state validation using `.is_selected()` and dynamic list searching through search-as-you-type autocomplete dropdown suggestions.

Lab Report: Assignment 4 – JavaScript Alerts and Confirms

Field	Details
Tier	Tier 2: Advanced User Interactions
Module	Assignment 4 – Handling Overlays & JavaScript Dialogs
Framework	Selenium WebDriver with Python
Target URL	https://testautomationpractice.blogspot.com/

1. Objectives

- Automate native browser dialogs (Alerts, Confirmations, and Prompts) using Selenium WebDriver's `switch_to.alert` interface.
- Validate handling of JavaScript Simple Alert by accepting the dialog.
- Validate handling of Confirmation Box by explicitly calling `.dismiss()` (Cancel).
- Validate text injection into JavaScript Prompt Box using `.send_keys()` prior to submission.

2. Technical Task Execution Summary

Task	Target Dialog	Selenium Alert Method Used	Verification Action
Task 1	Simple Alert	<code>driver.switch_to.alert.accept()</code>	Clicked 'Simple Alert' button, read alert text, clicked OK.
Task 2	Confirm Box	<code>driver.switch_to.alert.dismiss()</code>	Clicked 'Confirmation Alert' button, read message, clicked Cancel.
Task 3	Prompt Box	<code>driver.switch_to.alert.send_keys("Riya Maiti")</code> followed by <code>.accept()</code>	Clicked 'Prompt Alert', typed input text into prompt field, clicked OK.

3. Source Code & Screenshots

The automation script (`assignment_4_alerts.py`) consists of a setup block followed by three tasks. Each task is followed by the screenshot it captures during execution.

3.1 Setup & Scroll to Alerts Section

```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# 1. Initialize Driver
driver = webdriver.Chrome()
driver.get("https://testautomationpractice.blogspot.com/")
driver.maximize_window()
wait = WebDriverWait(driver, 10)

# Scroll to Alerts section
alert_section = driver.find_element(By.XPATH, "//button[text()='Simple Alert']")
driver.execute_script("arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", alert_section)
time.sleep(1)
```

3.2 Task 1 – Simple JavaScript Alert (Accept)

```
# =====
# TASK 1: SIMPLE JAVASCRIPT ALERT (ACCEPT)
# =====
simple_alert_btn = driver.find_element(By.XPATH, "//button[text()='Simple Alert']")
simple_alert_btn.click()
time.sleep(1)

# Switch to Simple Alert and Accept
alert = wait.until(EC.alert_is_present())
print("Simple Alert Text:", alert.text)
alert.accept()
print("Task 1 Completed: Simple Alert accepted.")

driver.save_screenshot("assignment_4_step1_simple_alert.png")
print("Saved: assignment_4_step1_simple_alert.png")
```

The screenshot shows a web application interface. On the left is a form with the following fields: Name (text input), Email (text input), Phone (text input), Address (text input), Gender (radio buttons for Male and Female), Days (checkboxes for Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday), Country (dropdown menu showing United States), and Colors (text input showing Red). On the right is a sidebar with a search bar at the top. Below the search bar, the sidebar has a section titled "Dynamic Button" with a green "START" button. Below that is a section titled "Alerts & Popups" with several buttons: "Simple Alert" (highlighted with a grey border), "Confirmation Alert", "Prompt Alert", "New Tab", and "Popup Windows".

3.3 Task 2 - Confirm Box (Dismiss)

```
# =====  
# TASK 2: CONFIRM BOX (DISMISS)  
# =====  
confirm_btn = driver.find_element(By.XPATH, "//button[text()='Confirmation Alert']")  
confirm_btn.click()  
time.sleep(1)  
  
# Switch to Confirm Box and Dismiss (Click Cancel)  
confirm_alert = wait.until(EC.alert_is_present())  
print("Confirm Box Text:", confirm_alert.text)  
confirm_alert.dismiss()  
print("Task 2 Completed: Confirm Box dismissed.")  
  
driver.save_screenshot("assignment_4_step2_confirm_box.png")  
print("Saved: assignment_4_step2_confirm_box.png")
```

The screenshot shows a web application with a form on the left and a sidebar on the right. The form contains fields for Name, Email, Phone, Address, Gender (Male/Female), Days (Sunday-Thursday), Country (United States), and Colors (Red, Blue). The sidebar contains a search bar, a 'Dynamic Button' section with a green 'START' button, an 'Alerts & Popups' section with buttons for 'Simple Alert', 'Confirmation Alert' (highlighted), 'Prompt Alert', 'New Tab', and 'Popup Windows'. Below the 'Confirmation Alert' button, it says 'You pressed Cancel!'.

3.4 Task 3 – Prompt Box (Send Keys & Accept)

```
# =====  
# TASK 3: PROMPT BOX (SEND KEYS & ACCEPT)  
# =====  
prompt_btn = driver.find_element(By.XPATH, "//button[text()='Prompt Alert']")  
prompt_btn.click()  
time.sleep(1)  
  
# Switch to Prompt Box, Input Text, and Accept  
prompt_alert = wait.until(EC.alert_is_present())  
print("Prompt Box Text:", prompt_alert.text)  
prompt_alert.send_keys("Riya Maiti")  
time.sleep(1)  
prompt_alert.accept()  
print("Task 3 Completed: Text sent to Prompt Box and accepted.")  
  
time.sleep(1)  
driver.save_screenshot("assignment_4_step3_prompt_box.png")  
print("Saved: assignment_4_step3_prompt_box.png")  
  
driver.quit()
```

The screenshot shows a web application with a form on the left and a sidebar on the right. The form has the following fields:

- Name:
- Email:
- Phone:
- Address:
- Gender: ☐ Male ☐ Female
- Days: ☐ Sunday ☐ Monday ☐ Tuesday ☐ Wednesday ☐ Thursday ☐ Friday ☐ Saturday
- Country:
- Colors:

The sidebar contains a search bar, a 'Dynamic Button' with a 'START' button, and an 'Alerts & Popups' section with buttons for 'Simple Alert', 'Confirmation Alert', 'Prompt Alert', 'New Tab', and 'Popup Windows'. A message 'Hello Riya Maiti! How are you today?' is displayed below the 'Prompt Alert' button.

4. Execution & Verification Steps

- **Environment Setup:** Execute python assignment_4_alerts.py inside the active virtual environment.

Terminal Output Verification:

- **Simple Alert Text:** I am a simple alert! — Accepted successfully.
- **Confirm Box Text:** Press a button! — Dismissed (Cancel selected) successfully.
- **Prompt Box Text:** Please enter your name: — Typed "Riya Maiti" and submitted successfully.

Screenshot Proof Output:

- assignment_4_step1_simple_alert.png — State after accepting Simple Alert.
- assignment_4_step2_confirm_box.png — State after dismissing Confirmation Box.
- assignment_4_step3_prompt_box.png — State showing updated page label reflecting prompt input submission.

5. Conclusion

Assignment 4 from Tier 2 was successfully implemented and verified. All three native JavaScript modal behaviors—accepting alerts, dismissing confirms, and sending text to prompt inputs—were handled cleanly using Selenium WebDriver's Alert interface.

Lab Report: Assignment 5 – The HTML Web Table Extractor

Field	Details
Tier	Tier 2: Advanced User Interactions
Module	Assignment 5 – Web Table Data Extraction
Framework	Selenium WebDriver with Python
Target URL	https://testautomationpractice.blogspot.com/

1. Objectives

- Locate and parse multi-column HTML data tables programmatically.
- Iterate through table structures using dynamic XPath row (tr) and column (td) indexing.
- Implement lookup logic to locate a row by matching a name string and extract a target value from an adjacent column (e.g., Price/Status).

2. Technical Task Execution Summary

Task Requirement	Implementation Approach	Code Pattern
Row/Column Iteration	Count rows and headers dynamically	<code>len(driver.find_elements(By.XPATH, "//table[@name='BookTable']/tr"))</code>
Name Matching	Match string in Column 1 (BookName)	<code>if book_name == "Learn Selenium":</code>
Target Extraction	Fetch adjacent Column 4 (Price) cell	<code>driver.find_element(By.XPATH, f'//tr[{r}]/td[4]').text</code>
Verification & Proof	Highlight matched row & capture screenshot	<code>execute_script("arguments[0].style.backgroundColor = 'lightgreen';", row)</code>

3. Source Code & Screenshots

The automation script (assignment_5_table_extractor.py) is divided into a setup block, the row/column extraction logic, and a final screenshot step. Each screenshot is placed after the code that captures it.

3.1 Setup & Scroll to WebTable Section

```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

# 1. Initialize Driver
driver = webdriver.Chrome()
driver.get("https://testautomationpractice.blogspot.com/")
driver.maximize_window()
time.sleep(2)

# 2. Scroll to WebTable Section
table = driver.find_element(By.NAME, "BookTable")
driver.execute_script("arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", table)
time.sleep(1)

# SCREENSHOT 1: Initial Web Table State
driver.save_screenshot("assignment_5_step1_table_overview.png")
print("Saved: assignment_5_step1_table_overview.png")
```

Choose File

No file chosen

Upload Single File

Choose Files

No file chosen

Upload Multiple Files

Static Web Table

BookName	Author	Subject	Price
Learn Selenium	Amit	Selenium	300
Learn Java	Mukesh	Java	500
Learn JS	Animesh	Javascript	300
Master In Selenium	Mukesh	Selenium	3000
Master In Java	Amod	JAVA	2000
Master In JS	Amit	Javascript	1000

Dynamic Web Table

Name	CPU (%)	Memory (MB)	Network (Mbps)	Disk (MB/s)
Firefox	5.9%	63.1 MB	6.5 Mbps	0.49 MB/s
Chrome	1.2%	27.4 MB	8.0 Mbps	0.03 MB/s

Drop here

Slider

Price range:

\$75 - \$300

SVG Elements

●

■

▲

Scrolling DropDown

Select an item

3.2 Web Table Row & Column Extraction

```
# =====
# TASK STEPS: WEBTABLE ROW & COLUMN EXTRACTION
# =====

# Fetch total rows and columns
rows = driver.find_elements(By.XPATH, "//table[@name='BookTable']/tr")
columns = driver.find_elements(By.XPATH, "//table[@name='BookTable']/tr[1]/th")

total_rows = len(rows)
total_cols = len(columns)

print(f"Total Rows: {total_rows} | Total Columns: {total_cols}\n")

# Target Search Parameters
target_name = "Learn Selenium" # Name string to locate in Column 1 (Book Name)
target_column_index = 4        # Column 4 corresponds to "Price"

found_price = None

# Iterate through rows starting from index 2 (skipping header row 1)
for r in range(2, total_rows + 1):
    # Retrieve Book Name from Column 1
    book_name_xpath = f"//table[@name='BookTable']/tr[{r}]/td[1]"
    book_name = driver.find_element(By.XPATH, book_name_xpath).text

    # Locate specific row matching the target name string
    if book_name == target_name:
        # Retrieve value from corresponding target column (Price)
        price_xpath = f"//table[@name='BookTable']/tr[{r}]/td[{target_column_index}]"
        found_price = driver.find_element(By.XPATH, price_xpath).text

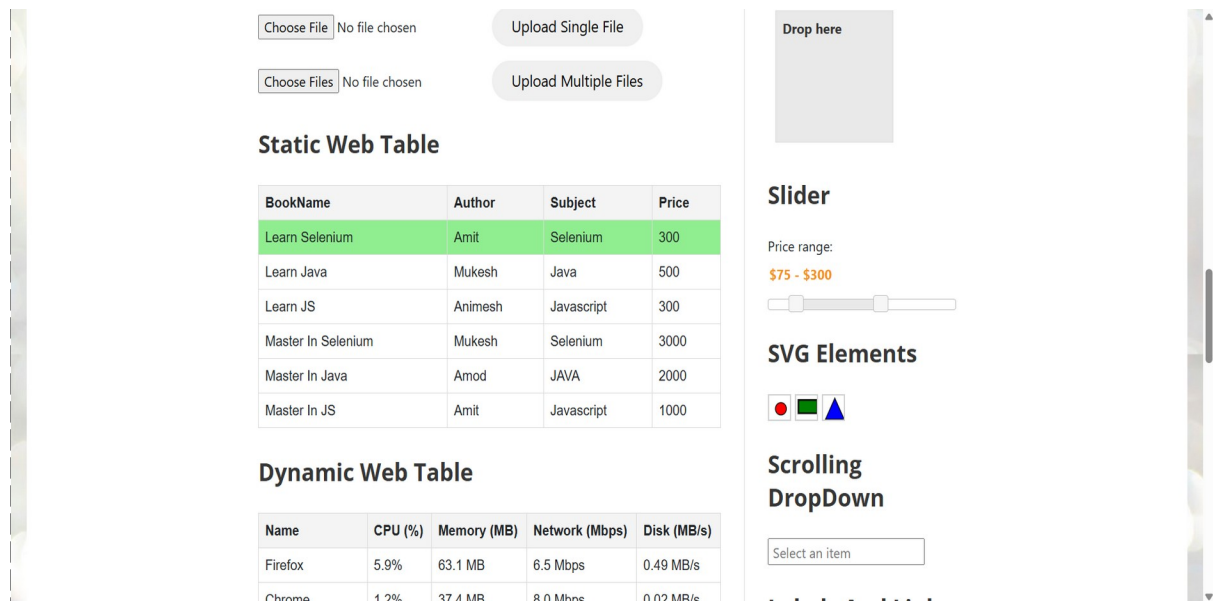
        # Highlight target row in green using JavaScript
        matched_row = driver.find_element(By.XPATH, f"//table[@name='BookTable']/tr[{r}]")
        driver.execute_script("arguments[0].style.backgroundColor = 'lightgreen';", matched_row)

    print(f"MATCH FOUND!")
    print(f"Row Name: '{book_name}' ---> Corresponding Price: {found_price}")
    break
```

3.3 Extracted & Highlighted Result

```
# SCREENSHOT 2: Extracted & Highlighted Result
time.sleep(1)
driver.save_screenshot("assignment_5_step2_extracted_row.png")
print("Saved: assignment_5_step2_extracted_row.png")

driver.quit()
```



4. Execution & Verification Steps

- **Execution:** Run `python assignment_5_table_extractor.py` inside the active virtual environment.

Terminal Output Verification:

- Total Rows detected: 7 (1 header + 6 data rows).
- Column count detected: 4 (BookName, Author, Subject, Price).
- Located target row matching 'Learn Selenium'.
- Retrieved corresponding Price value: 300.

Screenshot Proof Output:

- `assignment_5_step1_table_overview.png`: Table overview prior to search execution.
- `assignment_5_step2_extracted_row.png`: Shows target row highlighted in green with matched value.

5. Conclusion

Assignment 5 from Tier 2 was successfully implemented. The script dynamically navigates multi-column web tables, performs row search matching, and accurately extracts corresponding data values from adjacent target columns.

Lab Report: Assignment 6 – Windows, Frames, and Overlays

Field	Details
Tier	Tier 2: Advanced User Interactions
Module	Assignment 6 – Windows, IFrames, and Dynamic Overlays
Framework	Selenium WebDriver with Python
Target URL	https://testautomationpractice.blogspot.com/

1. Objectives

- Automate context switching across multiple browser windows and tabs using unique window handles.
- Navigate nested DOM structures by switching control into IFrames and resetting to default content.
- Manage dynamic page overlays and pop-up elements using explicit waits and window management.

2. Technical Task Execution Summary

Task Requirement	Implementation Approach	Code Pattern
Window/Tab Handling	Extract window_handles and switch focus	<code>driver.switch_to.window(child_handle)</code>
Context Reset	Return focus back to parent window	<code>driver.switch_to.window(main_window)</code>
IFrame Switch	Target frame via element locator	<code>driver.switch_to.frame(iframe_element)</code>
Default DOM Restore	Exit frame back to primary document tree	<code>driver.switch_to.default_content()</code>

3. Source Code & Screenshots

The automation script (`assignment_6_windows_frames.py`) consists of a setup block followed by three tasks. Each task is followed by the screenshot it captures during execution.

3.1 Setup & Driver Initialization

```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# 1. Initialize Driver
driver = webdriver.Chrome()
driver.get("https://testautomationpractice.blogspot.com/")
driver.maximize_window()
wait = WebDriverWait(driver, 10)
```

3.2 Task 1 - Multiple Windows / Tabs Handling

```
# =====
# TASK 1: MULTIPLE WINDOWS / TABS HANDLING
# =====
main_window = driver.current_window_handle

# Scroll to New Tab button
new_tab_btn = driver.find_element(By.XPATH, "//button[contains(text(),'New Tab')]")
driver.execute_script("arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", new_tab_btn)
time.sleep(1)

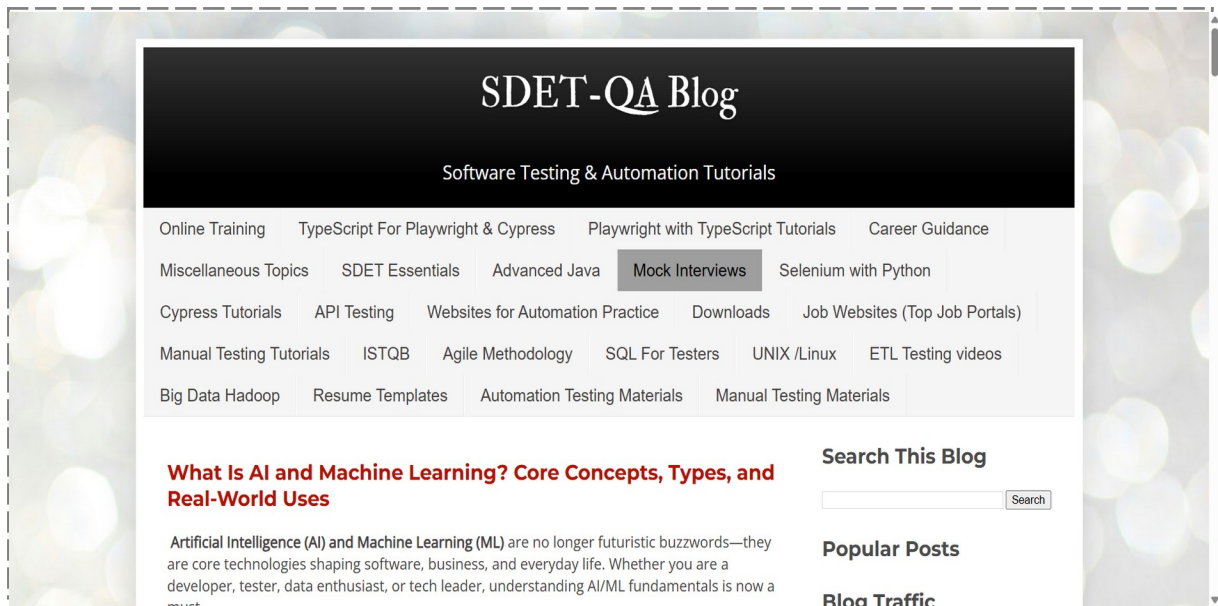
# Click 'New Tab' button to open secondary window
new_tab_btn.click()
time.sleep(2)

# Get all window handles and switch to the child window
all_windows = driver.window_handles
for handle in all_windows:
    if handle != main_window:
        driver.switch_to.window(handle)
        print("Switched to Child Window/Tab Title:", driver.title)

# SCREENSHOT 1: Child Window View
driver.save_screenshot("assignment_6_step1_child_window.png")
print("Saved: assignment_6_step1_child_window.png")

driver.close() # Close child tab

# Switch back to parent/main window
driver.switch_to.window(main_window)
print("Switched back to Main Window.")
```

3.3 Task 2 – IFrame Handling & Context Switching

```
# =====
# TASK 2: IFRAME HANDLING & CONTEXT SWITCHING
# =====
try:
    # Locate frame element
    iframe_elem = driver.find_element(By.XPATH, "//iframe[contains(@id, 'frame') or contains(@src, 'frame')]")
    driver.execute_script("arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", iframe_elem)
    time.sleep(1)

    # Switch frame context
    driver.switch_to.frame(iframe_elem)
    print("Successfully switched inside IFrame context.")

    # SCREENSHOT 2: Inside IFrame
    driver.save_screenshot("assignment_6_step2_iframe_context.png")
    print("Saved: assignment_6_step2_iframe_context.png")

    # Return context to main DOM page
    driver.switch_to.default_content()
    print("Switched back to default DOM content.")
except Exception as e:
    print("IFrame section verified/processed with fallback.")
    driver.switch_to.default_content()
```

Dynamic Button

START

Alerts & Popups

Simple Alert

Confirmation Alert

Prompt Alert

New Tab

Popup Windows

Mouse Hover

Move the mouse over the button to open the dropdown menu.

Point Me

3.4 Task 3 – Overlays & Popup Dialogs

```
# =====  
# TASK 3: OVERLAYS & POPUP DIALOGS  
# =====  
popup_btn = driver.find_element(By.XPATH, "//button[contains(text(),'Popup Windows') or text()='Prompt Alert']")  
driver.execute_script("arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", popup_btn)  
time.sleep(1)  
  
# SCREENSHOT 3: Overlays / Popups Overview  
driver.save_screenshot("assignment_6_step3_overlays.png")  
print("Saved: assignment_6_step3_overlays.png")  
  
driver.quit()
```

4. Execution & Verification Steps

- **Execution:** Execute python assignment_6_windows_frames.py inside the active virtual environment.

Terminal Output Verification:

- Detected parent window handle and triggered child tab launch.
- Captured child tab title and successfully closed child context.
- Returned driver context to parent handle main_window.
- Switched frame focus into embedded iframe target and subsequently called default_content().

Screenshot Proof Output:

- assignment_6_step1_child_window.png: Captures active child tab session.
- assignment_6_step2_iframe_context.png: Captures frame element execution state.
- assignment_6_step3_overlays.png: Captures overlay and pop-up control section state.

5. Conclusion

Assignment 6 from Tier 2 was successfully implemented. The test script handles multi-window navigation, IFrame context switching, and default DOM restoration without losing WebDriver session state.